Lab 5: Static Code Analysis

Name: Likhith S
SRN: PES1UG24CS720
Section: L

Objective

The objective of this lab is to improve the quality, security and style of a Python program using three static analysis tools: Pylint, Bandit and Flake8. Static analysis examines source code without running it, so problems can be found and fixed before the program is ever executed. The script assigned to me was loan_calculator.py from the repository SETAPESU26/L5_57_loan-eligibility.

Tools and Environment

- **Pylint 4.1.3:** a strict code reviewer. It reports logical errors, unused code, poor design and naming problems, and gives the code a score out of 10.
- **Bandit 1.9.4:** a security scanner. It reports dangerous functions and insecure coding patterns, each with a severity and a confidence level.
- **Flake8 7.4.1:** a style checker. It enforces PEP 8 rules such as spacing, line length and unused imports.
- Python 3.13.7 on macOS, inside a virtual environment. One extra package, paramiko 5.0.0, was installed as part of the security fixes.

The Program Analysed

loan_calculator.py is a small loan toolkit of about 90 lines. It calculates the monthly EMI of a loan, checks whether an applicant is eligible, finds the maximum loan an income supports, builds an amortisation schedule, assigns a risk band to a credit score, and estimates the interest saved by prepaying. It can also save and load loan applications from a file, upload a report to a credit bureau, and email an applicant about their status. Before any changes, the program ran correctly and printed its demonstration output, which gave a baseline to compare against after the fixes.

Procedure

The environment was set up and the three tools were run on the original script, with each report saved in a before folder. The same commands were run again after the fixes, with reports saved in an after folder.

```
python3 -m venv venv
source venv/bin/activate
pip install pylint bandit flake8
python3 loan_calculator.py
pylint loan_calculator.py > before/pylint_report.txt
bandit -r loan_calculator.py > before/bandit_report.txt
flake8 loan_calculator.py > before/flake8_report.txt
```

The fixes were made in three passes, one per tool, in the order Bandit, Pylint, Flake8. Each pass was verified by re-running the tools and then committed separately. Bandit was done first because its fixes change behaviour the most, so keeping them in their own commit makes them easy to review.

Results Summary

Tool	Before	After
Pylint	5.17 / 10 (43 messages)	10.00 / 10
Bandit	8 issues (2 High, 1 Medium, 5 Low)	0 issues
Flake8	22 issues	0 issues (empty report)

The Pylint target in the handout was a score above 9.5, and the final score is 10.00. Bandit has no remaining findings at any severity, and Flake8 produces no output at all.

Issue Table

Line numbers refer to the original script. Issues that one fix resolved are grouped together.

Issue	Type	Line(s)	Description	Fix Approach
Insecure FTP (Bandit B402, B321; High)	Security	3, 68	ftplib upload sends credentials and data unencrypted	Replaced with SFTP using paramiko
Unsafe deserialization (Bandit B302; Medium)	Security	4, 57, 64	marshal.loads on file contents can run untrusted data	Switched storage to JSON (loans.json)
Hardcoded password (Bandit B107; Low)	Security	67	Password "Bureau#2024" used as a default argument	Read from the BUREAU_PASSWORD environment variable
Weak random generator (Bandit B311; Low)	Security	51	random.randint used to generate application IDs	Used secrets.randbelow
Subprocess use (Bandit B404, B603, B607; Pylint W1510)	Security	6, 74	mail command run through subprocess with a user-supplied address	Replaced with smtplib and EmailMessage
Mutable default argument (Pylint W0102, R0913, R0917)	Bug	20	history=[] is shared across calls and never read; 6 arguments	Removed the unused parameter
Unclosed files (Pylint R1732)	Bug	56, 63, 70	Files opened and closed by hand	Used with open(...)
Global statement (Pylint W0603)	Design	62	load_applications rebinds a module-level variable	Update the dictionary in place
Shadowed built-in and unused code (Pylint W0622, W0612, W0613)	Code smell	33 to 38	sum shadowed; month and annual_rate unused	Replaced loop with one multiplication; removed argument
Simplifiable logic (Pylint R1703, R1705, R1714, C0325)	Refactor	28, 79, 82	Chained ==, else/elif after return, redundant parentheses	Used an in test and direct returns
Naming (Pylint C0103)	Style	11 functions	camelCase function names	Renamed to snake_case
Missing docstrings (Pylint C0114, C0116)	Style	Module and 12 functions	No docstrings	Added one-line docstrings

Issue	Type	Line(s)	Description	Fix Approach
Unused import and empty f-string (Flake8 F401, F541; Pylint W0611, W1309)	Style	2, 104	math never used; f-string without placeholders	Removed both
Blank lines (Flake8 E302, E305)	Style	14 to 106	One blank line between top-level definitions	Two blank lines
Line length (Flake8 E501; Pylint C0301)	Style	20, 55, 70, 78, 95, 96, 100	Lines longer than 79 characters	Wrapped the lines

Fixes in Detail

Bandit commit. The two highest-severity findings were the FTP import and call. FTP sends the username, password and file contents in plain text, and Bandit flags the import itself, so using the secure FTP variant would not have cleared it. The upload was rewritten to use SFTP through paramiko. The hardcoded bureau password was removed from the source and is now read from an environment variable, with an error raised if it is not set. The medium-severity marshal.loads call was replaced by JSON, which can only hold data and cannot execute code when loaded. Application IDs now come from the secrets module, and the mail command run through subprocess was replaced by smtplib with EmailMessage, which also rejects line breaks in headers.

```
# before
ftp = ftplib.FTP(BUREAU_HOST)
ftp.login(user, password)
# after
password = os.environ.get("BUREAU_PASSWORD")
transport = paramiko.Transport((BUREAU_HOST, 22))
transport.connect(username=user, password=password)
```

Pylint commit. Pylint reported 43 messages. Most were naming (11 camelCase functions) and missing docstrings (13), which were fixed by renaming every function and its calls together and adding a one-line docstring to each function and to the module. The real code problems were the mutable default argument history=[], which kept growing across calls and was never read, unused variables and arguments, a shadowed built-in (sum), manual file handling without with, and a global statement. The history parameter was removed, which also cleared the too-many-arguments warning. The unused annual_rate argument was removed from max_loan_amount, whose loop became a single multiplication with the same result. The unused math import and the f-string without placeholders were also fixed here because both tools reported them.

```
# before
if loan_type == "home" or loan_type == "car" or loan_type == "personal":
    return True
else:
    return False
# after
return loan_type in ("home", "car", "personal")
```

Flake8 commit. Only formatting issues remained: one blank line instead of two between top-level definitions (E302, E305), and four lines longer than 79 characters (E501). The blank lines were fixed with autopep8 restricted to those two codes, so it could not change anything else. The four long lines were wrapped by hand.

Verification

After each pass the tools were re-run and the program was executed again. The program output after all fixes matches the output before, apart from the randomly generated application ID. The final reports are saved in the after folder, and the Flake8 report is empty.

Commit	Description
c01c471	Add original script before static analysis
c84c65f	Add before static analysis reports
750cdc3	Fix Bandit findings (FTP, marshal, hardcoded password, PRNG, subprocess)
a5c37cd	Fix Pylint findings (naming, docstrings, mutable default, unused code, simplified logic)
7760ccc	Fix Flake8 findings (blank lines, line length)
00066ba	Add after static analysis reports

Observations and Limitations

- **Some fixes changed behaviour.** The history and annual_rate parameters no longer exist, saved applications are now stored in loans.json instead of loans.bin, uploading requires paramiko and the BUREAU_PASSWORD variable, and notifications use a local SMTP server. Nothing in the demonstration called the removed parameters, but a real caller would need updating.
- **The tools did not find the logic bugs.** The demonstration passes an interest rate of 9 instead of 0.09, so the EMI for a 5 lakh loan prints as 375000.00 instead of about 10,400. An unknown loan type such as "crypto" is approved, because a high income returns True before the loan type is checked, and a high income or credit score also skips the FOIR check. These are business-logic errors, so they were left unchanged, since this lab covers only what the tools report.
- **Bandit has blind spots.** The new SFTP code does not verify the server host key, which a production system should. Bandit does not report this.
- **A perfect score is not correctness.** A Pylint score of 10 and empty Bandit and Flake8 reports show the code follows the tools' rules. They do not show that the program does the right thing.

Reflection

Purpose of static analysis. Static analysis finds a whole class of problems cheaply and early, without needing test inputs or a running system. A hardcoded password or an unclosed file is easy to miss in a code review but is found instantly by a tool.

Using the three tools together. Each tool checks a different thing, and their findings only partly overlap. Bandit found the security risks that neither of the others looks for. Pylint found the bugs and design problems, such as the shared mutable default argument. Flake8 found only style problems, but consistent style makes the code easier for others to read and review.

Reading severity. Severity decided the order of work. The two High Bandit findings and the Medium finding were fixed first, because they are real risks. Naming and spacing issues were left to the end, since they do not change how the program behaves. Confidence also matters: a finding with low confidence needs a manual check before acting on it.

Effect on maintainability. After the fixes the code has docstrings, consistent names, no shared state between calls, no leaked file handles and no secrets in the source. Each of these makes the code easier to understand and safer to change. Keeping one commit per tool also left a clear history of what was changed and why.

Conclusion

Using Pylint, Bandit and Flake8 on loan_calculator.py raised the Pylint score from 5.17 to 10.00 and cleared all Bandit and Flake8 findings, with the program output unchanged. The lab showed that static analysis is a fast and effective way to catch security, quality and style problems, and also that it complements testing and code review but does not replace them.